certora

Security Assessment

Polymarket V2

April 2026

Prepared for Polymarket

certora

Table of Contents

Project Summary **3**
- Project Scope 3
- Project Overview 3
- Protocol Overview 4

Assessment Methodology **5**

Threat and Security Overview **6**
- Findings Summary 9
- Severity Matrix 9

Detailed Findings **10**
- Medium Severity Issues 12
  - M-01: Bridging will often revert due to wrong refund logic 12
  - M-02: Missing checks upon bridging allow unintended state changes 13
  - M-03: Initiator is not provided in horizontal callbacks 14
- Low Severity Issues 15
  - L-01: CCIP bridging fee overpayments could be lost due to non-atomical quoting 15
  - L-02: Order matching could be subject to gas griefing 16
  - L-03: Data type mismatch in OracleAggregator prevents resolution of markets with > 256 conditions 18
  - L-04: `convert` doesn't support callbacks like the other asset-transferring functions 19
  - L-05: Migrations can revert due to insufficient balance upon merges 20
- Informational Severity Issues 22
  - I-01: Unused variables, errors and not completed TODOs 22
  - I-02: Unnecessary USDCe approval inherited in NegRiskCtfCollateralAdapter 23

Disclaimer **24**

About Certora **24**

certora

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Polymarket V2	https://github.com/Polymarket/polymarket-v2	d6ebf3e (initial) a340369 (final)	Solidity

Project Overview

This document describes the security review of **Polymarket V2**. The work was undertaken from **March 23rd, 2026** to **April 17th, 2026**.

The following contract list is included in our scope:

```
src/abstract/*
src/adapters/*
src/auth/*
src/bridge/ (Everything except the ones listed not in scope below)
src/collateral/*
src/exchange/*
src/libraries/*
src/modules/*
src/oracle/ (Everything except the ones listed not in scope below)
src/positionManager/*
src/routers/*
src/utils/*
```

Not in scope:

```
src/bridge/CcipBridge.sol
src/bridge/LzBridge.sol
```

```
src/dev/*
src/external/*
src/legacy/*
src/mocks/*
src/oracle/modules/arbitrators/QuorumArbitratorModule
src/oracle/modules/disputers/EOADisputerModule
src/oracle/modules/reporters/EOAReporterModule
src/test/*
src/*/test/ (Basically any test folder)
src/*/dev/ (Basically any dev folder which does test setup)
src/*/mocks/ (Basically any mocks folder for tests)
```

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Polymarket V2 is a modular prediction market protocol built on Solidity 0.8.34 with Foundry and Solady. It manages prediction market positions as ERC-1155 tokens through a central `PositionManager` that routes to pluggable market modules (`BinaryModule` and `NegRiskModule`).

Positions use structured IDs that encode all routing information directly in the token ID (`moduleId`, `baseHash`, `conditionIndex`, `outcomeIndex`), eliminating storage lookups for routing. An `OracleAggregator` provides a modular oracle framework with pluggable reporter, disputer, and arbitrator modules. An `Exchange` contract handles EIP-712 signed order matching for position trading. A `BridgeBase` abstract contract enables cross-chain position and collateral transfers.

Core operations (split, merge, redeem) follow an effects-first callback pattern: the module mints output tokens, calls back to the sender to pull input tokens, then verifies the input arrived. The `Router` uses a zero-callback variant where assets are pre-transferred before calling the module. This callback pattern intentionally enables flash-loan-like behavior — callers receive minted tokens before paying for them.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

certora

Threat and Security Overview

Polymarket V2 is a modular prediction market system built on a central ERC1155 token ledger. The architecture separates asset ownership from market logic, enabling extensible market types while maintaining a unified collateral and settlement infrastructure.

System Level Model and Security Elements

The system follows a hub-and-spoke model where the PositionManager acts as the hub for all position tokens.

- **PositionManager**: The core ERC1155 token that mints/burns position tokens. It delegates logic to modules via a routing system encoded in the token IDs.
- **Modules**: Specialized contracts (Binary, NegRisk) that define market creation, splitting, merging, and redemption logic.
- **CollateralToken (PMCT)**: An ERC20 wrapper for underlying assets like USDC, featuring onramp/offramp mechanisms.
- **OracleAggregator**: A singleton contract managing result reporting and disputes through a modular reporter system.
- **Routers**: Stateless entry points that handle complex multi-step operations for end-users.
- **Exchange**: Matches taker and maker orders.

Expected Invariants and Trust Boundaries

The protocol relies on several critical invariants to ensure solvency and correctness:

- **Solvency Invariant**: The total collateral must always be sufficient to back all outstanding position tokens at their maximum possible redemption value.
- **Position ID Uniqueness**: Token IDs must uniquely and immutably encode their module, baseHash, condition, and outcome to prevent cross-market collisions.
- **Redemption Integrity**: For any condition, exactly one unit of collateral is redeemable per winning position set, and the sum of winning outcomes must equal the total collateral deposited.

Trust Boundaries:

- **Owner/Admin**: Trusted to authorize upgrades (UUPS) and register valid modules.
- **Modules**: Trusted by the `PositionManager` to mint/burn tokens correctly. A malicious module could break the solvency invariant for the entire system.
- **Oracles**: The `OracleAggregator` trusts its reporters to provide accurate market resolutions, though a dispute mechanism exists to mitigate EOA reporter risk.

Security Assumptions and Attack Vectors

Security Assumptions:

- The system assumes that only "safe" modules are registered by the owner.
- It assumes underlying collateral (USDC and USDC.e) maintains its peg and follows standard ERC20 behavior.
- Operators behave in good faith - initializes state correctly, matches taker and maker orders as expected without misbehaving.

Attack Vectors:

- **Economic Risks**: Manipulation of market resolution via oracle disputes or exploiting fee models in the Exchange.
- **Access Rights**: Unauthorized upgrades or role assignments through the `InitializableRoles` system.
- **Dependency Surfaces**: Risks inherited from external integrations like LayerZero V2 or Chainlink CCIP for cross-chain bridging. Also using external libraries from Solady.
- **Cross-Protocol Interactions**: Reentrancy during callbacks (split/merge) or exploitation of the "zero-callback" pattern in the Router.
- **Native Token Refund Denial of Service**: The `BridgeRouter` lack of a `receive()` function causes LayerZero bridging transactions to revert when native token gas refunds are sent to the router instead of the user. (M-01)
- **Bridging non-existent positions to DoS condition preparation**: Missing check in the bridge allows an attacker to bridge non-existent positions and DoS legacy condition preparation. (M-02)
- **Horizontal Callback Authentication Spoofing**: `NegRiskModule` horizontal callbacks omit the `initiator` address, preventing receivers from verifying the source and allowing attackers to potentially trigger unauthorized collateral drains. (M-03)

- **Assembly Logic Errors in `unsafeTransfer`**: Manual memory management and stack manipulation in the `PositionManager`'s custom ERC1155 transfer functions could bypass standard balance checks if implemented incorrectly.
- **ERC-1271 Return Bomb Gas Griefing**: The Exchange's signature validation fails to limit return data size, allowing malicious signing contracts to consume an operator's entire gas budget via large memory payloads. (L-02)

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	3	3	3
Low	5	5	4
Informational	2	2	1
Total	10	10	8

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
M-01	Bridging will often revert due to wrong refund logic	Medium	Fixed
M-02	Missing checks upon bridging allow unintended state changes	Medium	Fixed
M-03	Initiator is not provided in horizontal callbacks	Medium	Fixed
L-01	CCIP bridging fee overpayments could be lost due to non-atomical quoting	Low	Acknowledged
L-02	Order matching could be subject to gas griefing	Low	Fixed
L-03	Data type mismatch in OracleAggregator prevents resolution of markets with > 256 conditions	Low	Fixed
L-04	`convert` doesn't support callbacks like the other asset-transferring functions	Low	Fixed
L-05	Migrations can revert due to insufficient balance upon merges	Low	Fixed
I-01	Unused variables, errors and not completed TODOs	Informational	Fixed

I-02	Unnecessary USDCe approval inherited in NegRiskCtfCollateralAdapter	Informational	Acknowledged

Medium Severity Issues

M-01: Bridging will often revert due to wrong refund logic

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/routers/BridgeRouter.sol	Status: Fixed	

Description:

Upon bridging via LZ, a refund address is provided as the msg.sender where native tokens are refunded if there has been an overpayment. The issue is that if a user uses the typical flow which is routing his call via the BridgeRouter, then the msg.sender is the router contract and he will get the refund. As the router contract does not have functionality to receive native tokens directly (no receive function), then that will simply revert.

Note that there is a quoting function which can compute the bridging cost so that there will be no overpayment or underpayment. However, as this is not called atomically in any of the flows and it's a standalone function, then the cost can move between off-chain quoting and actual on-chain execution (LayerZero specifically notes this scenario as a possibility).

Recommendations:

Refactor the refunding logic based on whether the call is routing via the router or not.

Customer Response:

Fixed in PR#168.

Fix Review:

Fix confirmed.

M-02: Missing checks upon bridging allow unintended state changes

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/bridge/abstract/BridgeBase.sol	Status: Fixed	

Description:

The root cause of the issue is the fact that bridging positions does not check whether the condition is prepared. This allows multiple different attack paths and unintended state changes, one of them being:

1. Attacker is on a spoke chain, generating a new condition ID representation based on a legacy condition ID (a fully deterministic transition) that will be migrated.

2. Bridge a position based on that new condition ID via `BridgeBase::bridgePositions()`, targeting the hub and providing a 0 amount for the position. Send-side checks pass and this requires no existing positions.

3. Message is received on the hub, this prepares the condition there due to the `BaseModule(moduleAddr).prepareConditionFromBridge(conditionId);` line.

4. Whenever an actual preparation of a migration occurs, it will revert as preparing twice is not allowed. Subsequently, markets can not be resolved as `legacyConditionId` remains unset.

Recommendations:

Upon bridging positions, make sure that the condition ID is prepared. Furthermore, consider adding the same check upon splitting and merging positions (unless intended) as users can create “fake” positions that way.

Customer Response:

Fixed in PR#182.

Fix Review:

Fix confirmed.

M-03: Initiator is not provided in horizontal callbacks

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/modules/NegRiskModule.sol	Status: Fixed	

Description:

Upon regular splits/merges/redemptions, the initiator is provided in the callback receiver's callback function. However, for horizontal callbacks, this is not the case and simply the receiver (_to address) is provided. This is problematic as the function which triggers the callback can be called by anyone with an arbitrary callback receiver and arbitrary data. The initiator serves as an easy way for the callback receiver to authenticate the callback and check whether it has been initiated by someone trusted. However, since he is not provided there, anyone can spoof callbacks to the receiver and force him to take actions that he does not actually want (e.g. draining his collateral tokens to do a horizontal split).

Recommendations:

Change the negative risk interface to accept an initiator and instead of providing a _to address, provide the msg.sender in the NegRiskModule so the callback receiver can easily and properly authenticate the call.

Customer Response:

Fixed in PR#160.

Fix Review:

Fix confirmed.

Low Severity Issues

L-01: CCIP bridging fee overpayments could be lost due to non-atomical quoting

Severity: Low	Impact: Low	Likelihood: Low
Files: src/bridge/CcipBridge.sol#L148	Status: Acknowledged	

Description:

One of the bridges used on top of the BridgeBase is CcipBridge - as any other bridge it requires paying fees in order to bridge messages. An important distinction for CCIP is that the fees paid in excess are not refunded (in contrast to LZ bridge where the excess is refunded). Since quoting when bridging is done off-chain and then the calculated ETH gas amount is provided to the respective bridge function, it is possible that in the timeframe between off-chain quoting and on-chain transaction executions the fees might drop, causing overpayment. In the case of LZ this approach is ok as the extra funds are restored, however for CCIP if fees are not calculated atomically right before the ccipSend call, they might be in excess (as callers normally include buffers). For the bigger part of the cases it is expected the fees to be stable or the extra fee amounts to be small, but this tradeoff should still be taken into consideration.

Recommendations:

Consider if the tradeoff of using off-chain quoting with CCIP bridging is acceptable and if not, an atomical approach would be one way to handle it. Also it makes sense to add a note about this in the docs.

Customer Response:

Acknowledged. This is a CCIP transport / UX limitation rather than a protocol state bug. The CCIP router does not refund excess native fee, and our bridge forwards the full msg.value to CCIP, so callers must use quoteBridge*() close to execution and avoid overpaying. We accept this as an operational constraint of the current CCIP path.

L-02: Order matching could be subject to gas griefing

Severity: Low	Impact: Low	Likelihood: Low
Files: src/exchange/Exchange.sol#L1021	Status: Fixed	

Description:

The Exchange contract uses the _isValidERC1271 function to validate the order signatures of type POLY_1271. The concept behind such signatures is that they execute a staticcall to a target signing contract, which verifies it. The standard OZ contract SignatureChecker used to verify such types of signatures puts a restriction on the return data size to the first 32 bytes in order to check the magic value. However the custom verification done in _isValidERC1271, does not have such a check and loads the whole result into memory:

```
function _isValidERC1271(address signer, bytes32 hash, bytes calldata sig)
internal view returns (bool) {
        (bool success, bytes memory result) =
signer.staticcall(abi.encodeWithSelector(ERC1271_MAGIC_VALUE, hash, sig));
        return success && result.length >= 32 && abi.decode(result, (bytes4)) ==
ERC1271_MAGIC_VALUE;
    }
```

This enables the return bomb vulnerability, where the called contract can return an excessively big payload, which would consume all gas and revert the transaction. In this particular case it would lead to griefing of all the gas provided by the operator to process the array of orders. In addition it should be noted that gas griefing here is also possible through other means:

- infinite loops of the signer inside the static call
- hook enabled ERC1155 `POSITION_MANAGER.safeTransferFrom` throughout the different order matching stages

The likelihood of the described scenario is low as there is no economical incentive other than griefing the gas, but it still remains a valid case that is better addressed.

Recommendations:

Restrict the return data size from the signature verification static call and then on a more general level also consider the griefing scenarios that are some prevention mechanism like for example careful monitoring and filtration of orders on the BE side, before they get picked by the operator for execution

Customer Response:

Fixed in PR#186.

Fix Review:

Fix confirmed. Note that it's still possible for the maker to grief the order matching by exhausting all of the provided gas.

L-03: Data type mismatch in OracleAggregator prevents resolution of markets with > 256 conditions

Severity: Low	Impact: Low	Likelihood: Low
Files: src/oracle/OracleAggregator.sol#L65-L67	Status: Fixed	

Description:

The Position structure utilizes a uint16 data type for the conditionIndex, allowing an index limit of up to 65,535. The NegRiskModule aligns perfectly with this architectural design, correctly bounding the preparation of an event by enforcing that the number of conditions must be >= 2 and <= 65536.

However, the OracleAggregator::RequestConfig inconsistently defines the resultLength and subconditionCount fields as uint8. This data type mismatch caps the maximum processable condition limit at 255. Consequently, if an event is legitimately initialized and traded upon with 257 or more conditions - which is fully permitted and validated by the NegRiskModule - the OracleAggregator will be unable to resolve all of the conditions for that event.

Recommendations:

Update the resultLength and subconditionCount fields in OracleAggregator::RequestConfig to be of type uint16.

Customer Response:

Fixed in PR#196, PR#207 and PR#215.

Fix Review:

Fix confirmed.

L-04: `convert` doesn't support callbacks like the other asset-transferring functions

Severity: Low	Impact: Low	Likelihood: Low
Files: src/modules/NegRiskModule.sol#L262	Status: Fixed	

Description:

All of the asset-transferring functions like split, merge, redeem allow the user to use a callback contract to transfer the asset that needs to be burnt. However, the NegRiskModule::convert doesn't have this functionality and is therefore inconsistent with the rest of the protocol's architecture. An edge case that could cause problems here is if, for example, a user holds all of his assets (positions and collateral tokens) inside a callback contract without an option to arbitrarily transfer assets out. In this case, the user cannot use the convert functionality.

Recommendations:

Add a callback call in the NegRiskModule::convert to be consistent with the rest of the functions and allow a user to use a contract to transfer the position to the module.

Customer Response:

Fixed in PR#169.

Fix Review:

Fix confirmed.

L-05: Migrations can revert due to insufficient balance upon merges

Severity: Low	Impact: Low	Likelihood: Low
Files: src/modules/migration/BaseMigrationMixin.sol	Status: Fixed	

Description:

Migrations are used for legacy positions to be rolled over into their new V2 representation. For each migration, the other outcome is checked and potentially merged with the current position outcome, winding down the legacy positions. However, the logic is not perfect and causes reverts in the below case:

1. User wants to migrate `ConditionA`, he has 200 YES tokens and 100 NO tokens for that condition.

2. The private `_buildTransferAndMint()` transfers the legacy positions to the contract, now balances are 200 YES tokens and 100 NO tokens of the same condition (assuming no previous balances).

3. In the merging logic, we run 2 iterations. The first iteration merges the 100 of the YES tokens with 100 of the NO tokens, now balances are 100 YES tokens and 0 NO tokens. The second iteration handles the NO tokens and its amount value is 100, even though we have 0 NO tokens at this point. As its YES complementary has a balance of 100, the `complementaryAmount` field now has a value of 100. This causes a revert upon the merge as we have 0 NO tokens due to the first iteration even though the logic assumes we have a 100.

Recommendations:

There are multiple ways to fix the issue, we will recommend one. Consider adding a few lines which check the actual balance of the current position being handled and compare it with the amount variable of the current iteration, then change the amount variable to the lowest of these 2. Also consider adding a 0-amount check before merging to avoid 0-amount merges.

Customer Response:

Fixed in PR#170.

Fix Review:

Fix confirmed.

Informational Severity Issues

I-01: Unused variables, errors and not completed TODOs

Files:

src/oracle/modules/reporters/ChainlinkReporterModule.sol

Description:

The contracts define WindowNotEnded, InvalidAmountTransferred, UnresolvedCondition, InvalidYesOutcome, InsufficientBacking, InvalidEventId, EventNotBridged and MustPrepareViaEvent errors which are unused. Exchange has a maxFeeRate variable but it is not used. Contracts have some unresolved TODOs which can be found by `Ctrl/CMD + F` and searching for TODO:, e.g. PositionManager::uri().

Recommendations:

Consider resolving the above.

Customer Response:

Fixed.

Fix Review:

Fix confirmed.

I-02: Unnecessary USDCe approval inherited in NegRiskCtfCollateralAdapter

Description: The NegRiskCtfCollateralAdapter inherits from CtfCollateralAdapter. In the base CtfCollateralAdapter constructor, it executes:

```javascript
_usdce.safeApprove(_conditionalTokens, type(uint256).max);
```

For the base adapter, this is strictly required. The base adapter directly interacts with the Conditional Tokens Framework (CTF) to split, which means the CTF needs permission to pull USDCE from the adapter during CONDITIONAL_TOKENS.splitPosition() calls.

The NegRiskCtfCollateralAdapter overrides the internal _splitPosition, _mergePositions, and _redeemPositions functions.

Instead of interacting with the CTF directly, it delegates these actions to the NegRiskAdapter. When NegRiskAdapter executes a split, it pulls USDCE directly from the wrapper.

Because the NegRiskCtfCollateralAdapter overrides all paths that would have called the CTF's native split/merge functions, the wrapper will never command the CTF to pull USDCE. Therefore, the inherited max approval to CONDITIONAL_TOKENS lies completely dormant.

Recommendation: Consider overriding the approval to 0 in the constructor of NegRiskCtfCollateralAdapter to ensure there are no excess approvals lying around

Customer's response: Acknowledged.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.